

Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	XXXXXX
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used standard Python indentation (4 spaces).
2	Proper File Structure	Files and folders are logically organized	Yes	Project follows modular architecture with separate folders for routers, services, models, frontend and tests.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as config, emotion_pipeline, logger, result clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as data_prep, load_bilstm, load_bert, gemini_guidance use meaningful names.
5	Code Comments	Inline and block comments explain logic	Yes	Major sections and functionality are documented using comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend logic is divided into reusable service modules and API routers.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Reusable functions and modular architecture reduce code duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks are used in API requests and frontend operations.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Emotion Analyzer Service	Python (Transformers)	API routes and topic extraction	High
2	Fact Checker Service	Python (Google Gemini API)	Fact verification functionality	High
3	History Logger	Python (JSON)	Stores and retrieves conversation history	High
4	Feedback Logger	Python (JSON)	Stores user feedback from frontend	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Have proper indentation and clear folder structure.
Readability	5	Consists of proper readability and including comments for future use.
Reusability	5	Tech Stack and the resources can be used for future enhancements.
Documentation / Comments	5	Every phase of the project is clearly documented also comments are included where required.
Overall Score	5	The overall project follows a proper structure and maintains clarity for any future implementations